Boolean Algebra

Discrete Math, Fall 2025

Konstantin Chukharev

Boolean Algebra

“Мы почитаем всех нулями,
А единицами — себя.”

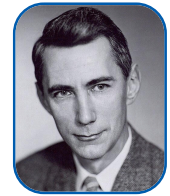
— А.С. Пушкин, «Евгений Онегин»

Gottfried Wilhelm Leibniz

George Boole

Augustus De Morgan

Charles Sanders Peirce



Claude Shannon

From Algebra to Digital Circuits

Historical note: George Boole's groundbreaking work *The Laws of Thought* (1854) created an algebraic system for logic a full century before the first electronic computers. He showed that logical reasoning could be captured by equations, replacing Aristotle's verbal arguments with mathematical symbols. This insight became the theoretical foundation for the digital age.

Key idea: Just as ordinary algebra works with numbers and operations like $+$ and $\times$, Boolean algebra works with **truth values** (0 and 1, or `false` and `true`) and operations like AND, OR, and NOT.

Historical milestones:

- **1703:** Leibniz describes binary arithmetic
- **1854:** Boole publishes algebraic logic
- **1937:** Shannon applies it to circuits
- **1940s:** First digital computers

Modern applications:

- Processor design (CPUs, GPUs)
- Formal verification (correctness proofs)
- Database optimization (queries)
- AI reasoning (SAT, planning)

Boolean Values: 0 and 1

In Boolean algebra, we work with exactly two values:

- **0** (`false`, off, low voltage)
- **1** (`true`, on, high voltage)

Example: In different contexts:

- **Mathematics:** "Is $x > 5$?" $\longrightarrow$ `true` (1) or `false` (0)
- **Programming:** `if (x && y)` $\longrightarrow$ evaluates to `true` or `false`
- **Circuits:** Wire voltage $\longrightarrow$ high (1) or low (0)
- **Sets:** Is element in set? $\longrightarrow$ yes (1) or no (0)

Why this matters: Modern computers represent all information using binary (0s and 1s).

Boolean algebra is the mathematics that makes digital computation possible.

What You Will Learn

In this lecture, you'll master Boolean algebra from foundations to applications:

1. Build Boolean expressions and evaluate them using **truth tables**
2. Prove identities using the **fundamental laws** (commutativity, De Morgan's laws, distributivity)
3. **Synthesize any Boolean function** from specifications using CNF, DNF, and Karnaugh maps
4. **Simplify complex expressions** through algebraic manipulation and graphical methods
5. Determine which operation sets are **functionally complete** using Post's criterion
6. **Design real digital circuits** with logic gates, flip-flops, and stateful elements
7. Apply Boolean algebra to **modern CS problems**: binary decision diagrams, SAT solving, and formal verification

Our goal: You'll gain both theoretical understanding (algebraic structure, formal proofs) and practical skills (circuit design, expression minimization). By the end, you'll see how the same mathematical framework powers everything from smartphone processors to AI planning algorithms.

Variables and Expressions

The Language of Boolean Logic

Just like natural languages have words and grammar rules, Boolean algebra has its own building blocks and composition rules.

Natural Language:

- Words: “cat”, “dog”
- Connectors: “and”, “or”, “not”
- Sentences: “Cat is black and dog is not white”

Boolean Algebra:

- Variables: x, y
- Operations: $\wedge$, $\vee$, $\neg$
- Expressions: $x \wedge \neg y$

Goal: Learn the “grammar” of Boolean logic — how to write and read Boolean expressions that represent logical relationships.

Boolean Variables: The Atoms

Definition 1: A *Boolean variable* is a variable that can take only one of two values: 0 (false) or 1 (true).

Think of Boolean variables as yes/no questions:

Example: Real-world Boolean variables (in programming):

- `is_logged_in` — Is the user logged in? (yes/no)
- `has_permission` — Does user have permission? (yes/no)
- `sensor_triggered` — Did the sensor activate? (yes/no)
- $x > 5$ — Is x greater than 5? (true/false)

Why binary? Digital circuits use voltage levels (high/low), making binary the natural choice for computation. Every piece of data in your computer is ultimately represented as 0s and 1s.

Boolean Operations: Connecting Ideas

We combine Boolean variables using operations (connectives) to express complex logic:

Definition 2 (Basic Operations):

- **NOT** ($\neg x$ or $\overline{x}$) — *reverses* the value
- **AND** ($x \wedge y$ or $x \cdot y$) — true only if *both* are true
- **OR** ($x \vee y$ or $x + y$) — true if *at least one* is true

Example: Access control logic:

- `is_admin OR has_permission` — User needs admin status OR explicit permission
- `is_logged_in AND NOT is_banned` — User must be logged in AND not banned
- `NOT (sensor1 AND sensor2)` — Not both sensors triggered simultaneously

Watch out: "OR" in Boolean logic is *inclusive* (can be both), unlike everyday speech where "coffee or tea" usually means "pick one."

Derived Operations: Shortcuts

More complex operations built from basic ones:

Definition 3 (Derived Operations):

- **XOR** ($x \oplus y$): true if *exactly one* is true
  - Formula: $(x \wedge \neg y) \vee (\neg x \wedge y)$
  - Example: "Dessert or coffee" (pick one, not both)
- **Implication** ($x \rightarrow y$): "if x then y"
  - Formula: $\neg x \vee y$
  - Example: "If it rains, bring umbrella"
- **Equivalence** ($x \Longleftrightarrow y$): true when values *match*
  - Formula: $(x \rightarrow y) \wedge (y \rightarrow x)$
  - Example: "Light is on if and only if switch is up"

Building Boolean Expressions

Definition 4: A *Boolean expression* is built recursively from variables and operations:

1. Variables (x, y, z) and constants ($0, 1$) are expressions
2. If f and g are expressions, so are: $\neg f$, $f \wedge g$, $f \vee g$, $f \oplus g$, $f \to g$, $f \iff g$

Example: Progressive complexity:

1. x — atomic variable
2. $\neg x$ — negation
3. $x \wedge y$ — two variables connected
4. $(x \wedge y) \vee z$ — nested expression
5. $\neg(x \vee (y \wedge z))$ — deeply nested
6. $(x \to y) \wedge (y \to z)$ — transitivity pattern

Key idea: Operations work on *any* expressions, not just variables. This compositionality lets us build arbitrarily complex formulas.

Truth Tables

Definition 5: A *truth table* is a complete list of all possible input combinations for a Boolean expression and their corresponding output values.

Note: For n Boolean variables, there are 2^n possible input combinations.

Example: Truth table for AND operation ($x \wedge y$):

x	y	$x \wedge y$
0	0	0
0	1	0
1	0	0
1	1	1

Truth Tables for Compound Expressions

We can build truth tables for complex expressions step by step.

Example: Truth table for $(x \wedge y) \vee (\neg x \wedge z)$:

x	y	z	$x \wedge y$	$\neg x \wedge z$	$(x \wedge y) \vee (\neg x \wedge z)$
0	0	0	0	0	0
0	0	1	0	1	1
0	1	0	0	0	0
0	1	1	0	1	1
1	0	0	0	0	0
1	0	1	0	0	0
1	1	0	1	0	1
1	1	1	1	0	1

Method: Build intermediate columns for subexpressions, then combine them for the final result.

Tautologies, Contradictions, Contingencies

Definition 6:

- A *tautology* is a Boolean expression that is always true (all 1s in output column)
- A *contradiction* is a Boolean expression that is always false (all 0s in output column)
- A *contingency* is a Boolean expression that is sometimes true and sometimes false

Example:

- **Tautology:** $x \vee \neg x$ (law of excluded middle)
- **Contradiction:** $x \wedge \neg x$ (law of non-contradiction)
- **Contingency:** $x \wedge y$ (depends on values of x and y)

Warning: In Boolean algebra, we typically care about expressions that are contingencies—those that represent actual functions. Tautologies and contradictions are constant functions.

Logical Equivalence

Definition 7: Two Boolean expressions f and g are *logically equivalent* (written $f \equiv g$) if they have identical truth tables—they produce the same output for every possible input combination.

Example: Show that $\neg(x \wedge y) \equiv \neg x \vee \neg y$ (De Morgan's law):

x	y	$\neg(x \wedge y)$	$\neg x \vee \neg y$	**Equal?**
0	0	1	1	✓
0	1	1	1	✓
1	0	1	1	✓
1	1	0	0	✓

Since all rows match, $\neg(x \wedge y) \equiv \neg x \vee \neg y$. ■

Evaluating Boolean Expressions

Given values for all variables, we can evaluate any Boolean expression.

Example: Evaluate $f(x, y, z) = (x \vee y) \wedge (\neg x \vee z)$ when $x = 1, y = 0, z = 1$:

$$\begin{aligned} f(1, 0, 1) &= (1 \vee 0) \wedge (\neg 1 \vee 1) \\ &= 1 \wedge (0 \vee 1) \\ &= 1 \wedge 1 \\ &= 1 \end{aligned}$$

Practice skill: Being able to quickly evaluate Boolean expressions is essential for debugging circuits and verifying logical designs.

Simplifying Boolean Expressions

Simple algebraic manipulation can simplify expressions:

Example: Simplify $f(x, y) = (x \wedge y) \vee (x \wedge \neg y)$:

$$\begin{aligned} f(x, y) &= (x \wedge y) \vee (x \wedge \neg y) \\ &= x \wedge (y \vee \neg y) && \text{(distributivity)} \\ &= x \wedge 1 && \text{(excluded middle)} \\ &= x && \text{(identity)} \end{aligned}$$

Example: Simplify $g(x, y, z) = (x \vee y) \wedge (x \vee \neg y)$:

$$\begin{aligned} g(x, y, z) &= (x \vee y) \wedge (x \vee \neg y) \\ &= x \vee (y \wedge \neg y) && \text{(distributivity)} \\ &= x \vee 0 && \text{(contradiction)} \\ &= x && \text{(identity)} \end{aligned}$$

Boolean Algebra Structure

From Expressions to Algebraic Structure

We have seen Boolean expressions as formulas. Now we study their *algebraic structure*: the operations and laws that govern how they behave.

Why formalize? Just as group theory abstracts the common structure of numbers, symmetries, and transformations, Boolean algebra abstracts the structure of logic, sets, and circuits.

What we already have:

- Variables and constants
- Operations: $\neg$, $\wedge$, $\vee$
- Expressions
- Truth tables

What we'll add:

- Formal axioms
- Fundamental laws
- Proof techniques
- Connection to lattices

Boolean Algebra: Formal Definition

Definition 8: A *Boolean algebra* is a set B with:

- Two binary operations: $\vee$ (join) and $\wedge$ (meet)
- One unary operation: $\neg$ (complement)
- Two special elements: 0 (bottom) and 1 (top)

satisfying the axioms on the next slide.

Example: The two-element Boolean algebra $\{0, 1\}$ with:

- $0 \vee 0 = 0, 0 \vee 1 = 1, 1 \vee 0 = 1, 1 \vee 1 = 1$
- $0 \wedge 0 = 0, 0 \wedge 1 = 0, 1 \wedge 0 = 0, 1 \wedge 1 = 1$
- $\neg 0 = 1, \neg 1 = 0$

Boolean Algebra Axioms

Definition 9 (Axioms of Boolean Algebra): For all $x, y, z \in B$:

1. **Commutativity:** $x \vee y = y \vee x$ and $x \wedge y = y \wedge x$
2. **Associativity:** $(x \vee y) \vee z = x \vee (y \vee z)$ and $(x \wedge y) \wedge z = x \wedge (y \wedge z)$
3. **Distributivity:** $x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$ and $x \vee (y \wedge z) = (x \vee y) \wedge (x \vee z)$
4. **Identity:** $x \vee 0 = x$ and $x \wedge 1 = x$
5. **Complement:** $x \vee \neg x = 1$ and $x \wedge \neg x = 0$

Note: Both $\wedge$ and $\vee$ distribute over each other — this is special to Boolean algebra (unlike ordinary arithmetic where only $\times$ distributes over $+$).

Examples of Boolean Algebras

Example (Power Set): For any set A, $(\mathcal{P}(A), \cup, \cap, \neg, \emptyset, A)$ is a Boolean algebra:

- Join: $\cup$ (union)
- Meet: $\cap$ (intersection)
- Complement: $X^c = A \setminus X$
- Bottom: $\emptyset$, Top: A

Example (Binary Strings): For n-bit binary strings with bitwise operations:

- Join: bitwise OR
- Meet: bitwise AND
- Complement: bitwise NOT
- Bottom: 000…0, Top: 111…1

Key insight: The same algebraic structure appears in logic, set theory, circuits, and more.

Fundamental Laws (Derived Properties)

From the axioms, we can prove many useful identities:

Theorem 1 (Basic Laws): For all $x, y \in B$:

1. **Idempotence:** $x \vee x = x$ and $x \wedge x = x$
2. **Absorption:** $x \vee (x \wedge y) = x$ and $x \wedge (x \vee y) = x$
3. **Null (Domination):** $x \vee 1 = 1$ and $x \wedge 0 = 0$
4. **Double Negation:** $\neg\neg x = x$
5. **Complement of Constants:** $\neg 0 = 1$ and $\neg 1 = 0$

Note: These can all be proven from the axioms using algebraic manipulation.

Proving Identities: Idempotence

Theorem 2: $x \vee x = x$ for all x in a Boolean algebra.

Proof:

$$\begin{aligned} x \vee x &= (x \vee x) \wedge 1 && \text{(identity)} \\ &= (x \vee x) \wedge (x \vee \neg x) && \text{(complement)} \\ &= x \vee (x \wedge \neg x) && \text{(distributivity)} \\ &= x \vee 0 && \text{(complement)} \\ &= x && \text{(identity)} \end{aligned}$$

$\square$

Method: Use axioms strategically — introduce 1 or 0, apply distributivity, then simplify.

Proving Identities: Absorption

Theorem 3: $x \vee (x \wedge y) = x$ for all x, y in a Boolean algebra.

Proof:

$$\begin{aligned} x \vee (x \wedge y) &= (x \wedge 1) \vee (x \wedge y) && \text{(identity)} \\ &= x \wedge (1 \vee y) && \text{(distributivity)} \\ &= x \wedge 1 && \text{(null law)} \\ &= x && \text{(identity)} \end{aligned}$$

$\square$

Example: Similarly, $x \wedge (x \vee y) = x$ by dual reasoning.

The Duality Principle

Theorem 4 (Duality Principle): If an identity holds in any Boolean algebra, then the *dual* identity (obtained by swapping $\vee \leftrightarrow \wedge$ and $0 \leftrightarrow 1$) also holds.

Example:

- Identity law: $x \vee 0 = x$, and its dual: $x \wedge 1 = x$
- Absorption: $x \vee (x \wedge y) = x$, and its dual: $x \wedge (x \vee y) = x$
- De Morgan: $\neg(x \vee y) = \neg x \wedge \neg y$, and its dual: $\neg(x \wedge y) = \neg x \vee \neg y$

Power of duality: Every theorem you prove gives you *two* theorems for free!

De Morgan's Laws

Theorem 5 (De Morgan's Laws): For all x, y in a Boolean algebra:

- $\neg(x \vee y) = \neg x \wedge \neg y$
- $\neg(x \wedge y) = \neg x \vee \neg y$

Proof: We prove $\neg(x \vee y) = \neg x \wedge \neg y$ by showing that $\neg x \wedge \neg y$ is the complement of $x \vee y$.

Need to verify:

1. $(x \vee y) \vee (\neg x \wedge \neg y) = 1$
2. $(x \vee y) \wedge (\neg x \wedge \neg y) = 0$

(Full proof left as exercise — use distributivity and complement laws.) $\square$

Common application: Negating complex conditions: `NOT (is_admin OR has_permission)` becomes `NOT is_admin AND NOT has_permission`.

Connection to Lattices

Definition 10: A *lattice* is a partially ordered set where any two elements x, y have:

- A *least upper bound* (join): $x \vee y$
- A *greatest lower bound* (meet): $x \wedge y$

Definition 11: A *bounded distributive lattice* is a lattice with bottom $\bot$, top $\top$, and distributivity:

$$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$$

Definition 12: A *complemented lattice* is a bounded lattice where every element x has a complement y such that $x \vee y = \top$ and $x \wedge y = \bot$.

Theorem 6: Every Boolean algebra is a complemented bounded distributive lattice.

Complement Uniqueness

Theorem 7: In a Boolean algebra, each element has a *unique* complement.

Proof: Suppose y and z are both complements of x. Then:

$$\begin{aligned}
y &= y \wedge 1 && \text{(identity)} \\
&= y \wedge (x \vee z) && (z \text{ is complement of } x) \\
&= (y \wedge x) \vee (y \wedge z) && \text{(distributivity)} \\
&= 0 \vee (y \wedge z) && (y \text{ is complement of } x) \\
&= (x \wedge z) \vee (y \wedge z) && (z \text{ is complement of } x) \\
&= (x \vee y) \wedge z && \text{(distributivity)} \\
&= 1 \wedge z && (y \text{ is complement of } x) \\
&= z && \text{(identity)}
\end{aligned}$$

Therefore $y = z$. $\square$

Summary of Boolean Algebra Structure

What we've established:

1. Boolean algebra is an algebraic structure with operations $\vee$, $\wedge$, $\neg$ and elements 0, 1
2. Five fundamental axioms govern the structure
3. Many useful laws (idempotence, absorption, De Morgan's) follow from the axioms
4. Duality principle: swap $\vee \leftrightarrow \wedge$ and $0 \leftrightarrow 1$ to get new theorems
5. Boolean algebras are complemented bounded distributive lattices
6. Complements are unique

Next: We'll use this algebraic structure to develop normal forms and minimization techniques.

Normal Forms

From Structure to Representation

We've established the algebraic structure of Boolean algebra.

Now: how do we *represent* any Boolean function systematically?

Goal: Find standard forms that can express *any* Boolean function. This lets us:

1. Synthesize circuits from specifications (truth tables)
2. Compare functions for equivalence
3. Systematically simplify expressions

What we have:

- Boolean algebra axioms
- Proof techniques
- Specific functions

What we need:

- Standard representations
- Construction algorithms
- Completeness guarantees

Building Blocks: Literals

Definition 13: A *literal* is either a Boolean variable or its negation.

Think of literals as the simplest meaningful statements:

Example: For variables x, y, z:

- **Positive literals:** x, y, z — "is true"
- **Negative literals:** $\neg x$, $\neg y$, $\neg z$ — "is false"

Example (Real-world interpretation):

- x might mean "user is logged in"
- $\neg x$ means "user is NOT logged in"
- The expression $(x \wedge \neg y) \vee z$ combines three literals: x, $\neg y$, and z

Key idea: Literals are the atoms of Boolean logic. Every complex expression is ultimately built from these simple building blocks using $\wedge$ and $\vee$.

Terms and Clauses

Definition 14:
- A *term* (or *product*) is a conjunction (AND) of literals
- A *clause* (or *sum*) is a disjunction (OR) of literals

Terms (Products):
- $x \wedge y$
- $x \wedge \neg y \wedge z$
- $\neg x \wedge \neg y \wedge \neg z$

"ALL of these must be true"

Clauses (Sums):
- $x \vee y$
- $x \vee \neg y \vee z$
- $\neg x \vee \neg y \vee \neg z$

"At least ONE must be true"

Example:
- Term $x \wedge \neg y \wedge z$ is true only when: $x = 1$, $y = 0$, $z = 1$
- Clause $x \vee \neg y \vee z$ is true when: $x = 1$ OR $y = 0$ OR $z = 1$ (or any combination)

Note: A single literal is both a 1-term and a 1-clause. Constants: 0 (empty term) and 1 (empty clause).

Disjunctive Normal Form (DNF)

Definition 15 (DNF): A Boolean formula is in *Disjunctive Normal Form (DNF)* if it is a disjunction (OR) of terms.

General form: $(t_1) \lor (t_2) \lor \ldots \lor (t_k)$ where each t_i is a term (conjunction of literals).

Example: DNF formulas:

- $(x \land y) \lor (\neg x \land z)$ — "(x and y) OR ($\neg x$ and z)"
- $(x \land \neg y \land z) \lor (\neg x \land y) \lor z$ — three alternatives
- $x \lor (y \land \neg z)$ — still DNF (mixed form)

Example: **NOT** in DNF:

- $(x \lor y) \land z$ — this is CNF (AND of ORs)
- $x \land (y \lor z)$ — OR nested inside AND violates DNF structure

Disjunctive Normal Form (DNF) [2]

Intuition: DNF says "the output is 1 if ANY of these scenarios happen," where each scenario is a specific combination of variable values.

Conjunctive Normal Form (CNF)

Definition 16 (CNF): A Boolean formula is in *Conjunctive Normal Form (CNF)* if it is a conjunction (AND) of clauses.

General form: $(c_1) \wedge (c_2) \wedge \ldots \wedge (c_k)$ where each c_i is a clause (disjunction of literals).

Example: CNF formulas:

- $(x \vee y) \wedge (\neg x \vee z)$ — both constraints must hold
- $(x \vee \neg y \vee z) \wedge (\neg x \vee y) \wedge z$ — three constraints
- $x \wedge (y \vee \neg z)$ — still CNF (mixed form)

Example: **NOT** in CNF:

- $(x \wedge y) \vee z$ — this is DNF (OR of ANDs)
- $x \vee (y \wedge z)$ — AND nested inside OR violates CNF structure

CNF vs DNF

Intuition: CNF says "ALL of these constraints must be satisfied," where each constraint offers multiple ways to be true.

DNF says "the output is 1 if ANY of these scenarios happen," where each scenario is a specific combination of variable values.

Duality: DNF and CNF are *dual* forms. Swap $\wedge$ and $\vee$ to convert between them.

Minterms and Maxterms

Definition 17: For n variables:

- A *minterm* is a term containing all n variables (each exactly once, positive or negated)
- A *maxterm* is a clause containing all n variables (each exactly once, positive or negated)

Example: For variables x, y, z:

- **Minterms:** $x \wedge y \wedge z$, $x \wedge y \wedge \neg z$, $x \wedge \neg y \wedge z$, ...
- **Maxterms:** $x \vee y \vee z$, $x \vee y \vee \neg z$, $x \vee \neg y \vee z$, ...

There are exactly $2^n = 2^3 = 8$ minterms and 8 maxterms for 3 variables.

Key property:

- Each minterm is 1 for **exactly ONE** input combination
- Each maxterm is 0 for **exactly ONE** input combination
- This makes them perfect building blocks for representing any function!

Minterm and Maxterm Indexing

We can index minterms and maxterms by their binary representations:

Example: For x, y, z (interpreting as bits: xyz):

Index	**Binary**	**Minterm m_i**	**Maxterm M_i**
0	000	$\neg x \land \neg y \land \neg z$	$x \lor y \lor z$
1	001	$\neg x \land \neg y \land z$	$x \lor y \lor \neg z$
2	010	$\neg x \land y \land \neg z$	$x \lor \neg y \lor z$
3	011	$\neg x \land y \land z$	$x \lor \neg y \lor \neg z$
4	100	$x \land \neg y \land \neg z$	$\neg x \lor y \lor z$
5	101	$x \land \neg y \land z$	$\neg x \lor y \lor \neg z$
6	110	$x \land y \land \neg z$	$\neg x \lor \neg y \lor z$
7	111	$x \land y \land z$	$\neg x \lor \neg y \lor \neg z$

Sum of Products (SoP)

Definition 18: A *sum of products (SoP)* (also called *canonical sum of minterms*) is a DNF where each term is a minterm.

General form: $f(x_1, ..., x_n) = \bigvee_{i \in I} m_i$ where $I \subseteq \{0, 1, ..., 2^n - 1\}$

Example: For the function with truth table:

x	y	z	f
0	0	0	0
0	0	1	**1**
0	1	0	0
0	1	1	**1**
1	0	0	0
1	0	1	0
1	1	0	**1**
1	1	1	**1**

Sum of Products (SoP) [2]

Algorithm: Pick rows where $f = 1$ (rows 1, 3, 6, 7):

$$\begin{aligned} f &= m_1 \lor m_3 \lor m_6 \lor m_7 \\ &= (\neg x \land \neg y \land z) \lor (\neg x \land y \land z) \lor (x \land y \land \neg z) \lor (x \land y \land z) \end{aligned}$$

Recipe: To build SoP, take each 1 in the truth table and OR the corresponding minterms.

Product of Sums (PoS)

Definition 19: A *product of sums (PoS)* (also called *canonical product of maxterms*) is a CNF where each clause is a maxterm.

General form: $f(x_1, ..., x_n) = \bigwedge_{i \in I} M_i$ where $I \subseteq \{0, 1, ..., 2^n - 1\}$

Example: For the **same function**, use rows where $f = 0$ (rows 0, 2, 4, 5):

$$\begin{aligned} f &= M_0 \wedge M_2 \wedge M_4 \wedge M_5 \\ &= (x \vee y \vee z) \wedge (x \vee \neg y \vee z) \wedge (\neg x \vee y \vee z) \wedge (\neg x \vee y \vee \neg z) \end{aligned}$$

Two approaches:

- SoP uses 1s in truth table (where function is true)
- PoS uses 0s in truth table (where function is false)
- Both represent the same function!

Why PoS? Sometimes PoS is more compact than SoP (when function has fewer 0s than 1s).

Completeness of Normal Forms

Theorem 8: Every Boolean function can be represented in both CNF and DNF.

Proof *Sketch for DNF (SoP)*: Given a truth table:

1. For each row where output is 1, create the corresponding minterm
2. OR all these minterms together
3. The result is the function in DNF (SoP form)

Why this works:

- Each minterm is 1 for exactly one input combination
- ORing them gives 1 exactly when the function should be 1
- This construction is always possible and always correct

□

Power of synthesis: Any Boolean function can be built from its truth table!

Shannon Expansion

Theorem 9 (Shannon Expansion): For any Boolean function f and variable x:

$$f(x, y_1, ..., y_n) = (\neg x \wedge f(0, y_1, ..., y_n)) \vee (x \wedge f(1, y_1, ..., y_n))$$

$$f(x, y_1, ..., y_n) = (x \vee f(0, y_1, ..., y_n)) \wedge (\neg x \vee f(1, y_1, ..., y_n))$$

Example: Expand $f(x, y) = x \oplus y$ by x:

$$\begin{aligned} f(x, y) &= (\neg x \wedge f(0, y)) \vee (x \wedge f(1, y)) \\ &= (\neg x \wedge (0 \oplus y)) \vee (x \wedge (1 \oplus y)) \\ &= (\neg x \wedge y) \vee (x \wedge \neg y) \end{aligned}$$

This gives us the standard XOR representation!

Converting Between Forms

Converting between DNF and CNF can be tricky:

Example (DNF to CNF using De Morgan's laws): Start with DNF: $(x \wedge y) \vee (\neg x \wedge z)$

Step-by-step:

1. Negate: $\neg((x \wedge y) \vee (\neg x \wedge z))$
2. Apply De Morgan: $(\neg(x \wedge y)) \wedge (\neg(\neg x \wedge z))$
3. Apply De Morgan again: $(\neg x \vee \neg y) \wedge (x \vee \neg z)$
4. Double negate to get back: $\neg\neg((\neg x \vee \neg y) \wedge (x \vee \neg z))$

Result is CNF: $(\neg x \vee \neg y) \wedge (x \vee \neg z)$

Warning: Direct algebraic conversion can cause exponential blowup in formula size! For complex functions, use Karnaugh maps or other minimization techniques.

Alternative: Build CNF/DNF directly from truth table using the SoP/PoS methods.

Summary: Canonical Forms

What we've learned:

1. Literals, terms, clauses are the building blocks
2. DNF: OR of terms (conditions that make function true)
3. CNF: AND of clauses (constraints that must all hold)
4. Minterms/maxterms: complete terms/clauses with all variables
5. SoP/PoS: canonical forms using minterms/maxterms
6. Shannon expansion: recursive decomposition by variables
7. Every Boolean function has CNF and DNF representations

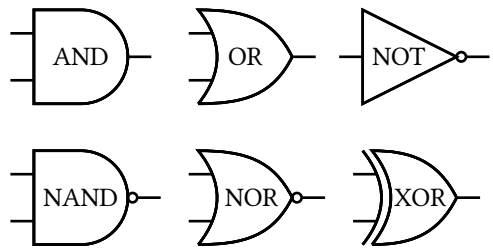
Next: Minimization techniques to reduce the size of these expressions (K-Maps, Quine-McCluskey).

Digital Circuits

Digital Logic Circuits

Definition 20: A *logic gate* is a physical device that implements a Boolean function, taking binary inputs and producing a binary output.

Gate	Formula	Description
AND	$A \wedge B$	Outputs 1 only when both inputs are 1
OR	$A \vee B$	Outputs 1 when at least one input is 1
NOT	$\neg A$	Outputs the opposite of the input
NAND	$A \overline{\wedge} B$	Outputs 0 only when both inputs are 1
NOR	$A \overline{\vee} B$	Outputs 0 when at least one input is 1
XOR	$A \oplus B$	Outputs 1 when inputs differ
XNOR	$A \equiv B$	Outputs 1 when inputs are the same



Note: NAND and NOR gates are *universal* — any Boolean function can be implemented using only NAND gates (or only NOR gates). For example, to implement AND using NAND:

$$A \wedge B = \neg\neg(A \wedge B) = \neg(A \overline{\wedge} B) = (A \overline{\wedge} B) \overline{\wedge} (A \overline{\wedge} B)$$

Combinational Logic

Definition 21: A *combinational circuit* is a circuit where the output depends only on the current input values, without any memory or state.

Example (Half Adder): Adds two single bits:

- Sum: $S = A \oplus B$
- Carry: $C = A \wedge B$

Example (Full Adder): Adds two bits plus a carry-in:

- Sum: $S = A \oplus B \oplus C_{\text{in}}$
- Carry-out: $C_{\text{out}} = (A \wedge B) \vee (C_{\text{in}} \wedge (A \oplus B))$

Sequential Logic and Memory

Definition 22: A *sequential circuit* is a circuit where the output depends on both current inputs and previous state (memory).

Example (Flip-Flops):

- **SR Latch**: Set-Reset memory element.
- **D Flip-Flop**: Data storage triggered by clock edge.
- **JK Flip-Flop**: Eliminates forbidden state of SR latch.
- **T Flip-Flop**: Toggle flip-flop for counters.